

Report di Laboratorio

Sfruttamento delle Vulnerabilita' XSS e SQL Injection sulla DVWA

Cybersecurity & Ethical Hacking

Studente	Lorenzo Ventanni
Corso	Cybersecurity & Ethical Hacking
Anno Accademico	2026
Esercizio	Vulnerabilita' XSS riflesse e SQL Injection su DVWA
Ambiente	Kali Linux 2026.1 + Metasploitable 2.0 (Oracle VirtualBox)
Data svolgimento	12 maggio 2026

1. Introduzione:

In questo laboratorio mi sono occupato dello sfruttamento di due tra le vulnerabilita' web piu' diffuse e storicamente piu' impattanti: il **Cross-Site Scripting (XSS)** di tipo riflesso e la **SQL Injection** in modalita' non-blind. L'obiettivo dell'esercitazione e' duplice: da un lato comprendere come queste falle nascano da una mancata o errata validazione dell'input lato server, dall'altro toccare con mano l'impatto concreto di un loro sfruttamento, dall'esecuzione di codice JavaScript arbitrario nel browser della vittima fino all'estrazione completa di credenziali dal database applicativo.

Come piattaforma di test ho utilizzato la **Damn Vulnerable Web Application (DVWA)**, ospitata sulla macchina vulnerabile **Metasploitable 2.0**, mentre come postazione di attacco ho impiegato la mia **Kali Linux 2026.1**. Le due macchine sono state configurate in modalita' bridged all'interno di Oracle VirtualBox, in modo da risiedere sulla stessa sottorete locale e poter comunicare in modo diretto, come accadrebbe in uno scenario reale di attacco da rete interna.

Il livello di sicurezza della DVWA e' stato impostato a **low**, scelta deliberata perche' lo scopo non e' bypassare difese ma osservare il funzionamento dei due attacchi nella loro forma piu' didattica e leggibile.

2. Obiettivi:

Obiettivo	Descrizione
Configurazione laboratorio	Predisporre l'ambiente virtuale in modo che Kali e Metasploitable comunichino su rete bridged, verificando la raggiungibilita' con ping.
Accesso alla DVWA	Raggiungere la DVWA dal browser di Kali e impostare il livello di sicurezza a LOW dal pannello DVWA Security.
XSS riflesso	Individuare il modulo XSS reflected della DVWA, costruire un payload JavaScript che venga riflesso ed eseguito nel browser della vittima.
SQL Injection manuale	Sfruttare un'iniezione SQL classica nel modulo SQL Injection per bypassare la logica della query (boolean-based) ed estrarre dati con UNION SELECT.
SQL Injection automatizzata	Replicare l'attacco precedente con sqlmap, dimostrando l'estrazione completa della tabella users e il successivo cracking degli hash MD5.
Documentazione	Produrre un report tecnico-didattico che descriva procedura, payload, evidenze (screenshot) e considerazioni finali.

3. Ambiente e Strumenti:

Componente	Configurazione
Hypervisor	Oracle VirtualBox - rete in modalita' bridged sulla sottorete 192.168.1.0/24.
Macchina attaccante	Kali Linux 2026.1 (192.168.1.164), kernel Linux.
Macchina vittima	Metasploitable 2.0 (192.168.1.170), Ubuntu legacy con servizi volutamente vulnerabili.
Applicazione bersaglio	Damn Vulnerable Web Application (DVWA) servita da Apache sulla porta 80, modulo SQL Injection e XSS reflected.
Browser	Mozilla Firefox di Kali, utilizzato per autenticarsi alla DVWA (utente admin) e inviare i payload.
Sqlmap	Versione 1.10.4#stable, eseguito da terminale Kali con cookie di sessione PHP della DVWA.

4. Procedimento:

Di seguito riporto la procedura seguita, suddivisa nei singoli step operativi. Per ciascuno indico i comandi eseguiti, l'output osservato e una breve interpretazione di cio' che e' accaduto.

STEP 1 | Verifica della connettivita' tra Kali e Metasploitable

Dopo aver avviato entrambe le macchine virtuali in VirtualBox con scheda di rete in modalita' **bridged**, ho verificato che le due VM ottenessero un IP sulla stessa LAN domestica (192.168.1.0/24). Da Kali ho lanciato un ping verso l'IP di Metasploitable e, simmetricamente, da Metasploitable un ping verso Kali, per confermare che il canale fosse bidirezionale.

```
kali@kali:~$ ping 192.168.1.170
msfadmin@metasploitable:~$ ip route
msfadmin@metasploitable:~$ ping 192.168.1.164
```

Tutti e dieci i pacchetti inviati da Kali hanno ricevuto risposta (0% packet loss), con RTT medio di circa 10 ms, valori coerenti con una rete locale. Il TTL=64 visibile nelle risposte conferma che dall'altra parte si trova un sistema Linux. Il comando `ip route` sulla Metasploitable ha mostrato la rotta di default via 192.168.1.1, ossia il gateway casalingo: ulteriore conferma che la VM e' realmente attestata sulla mia LAN fisica.

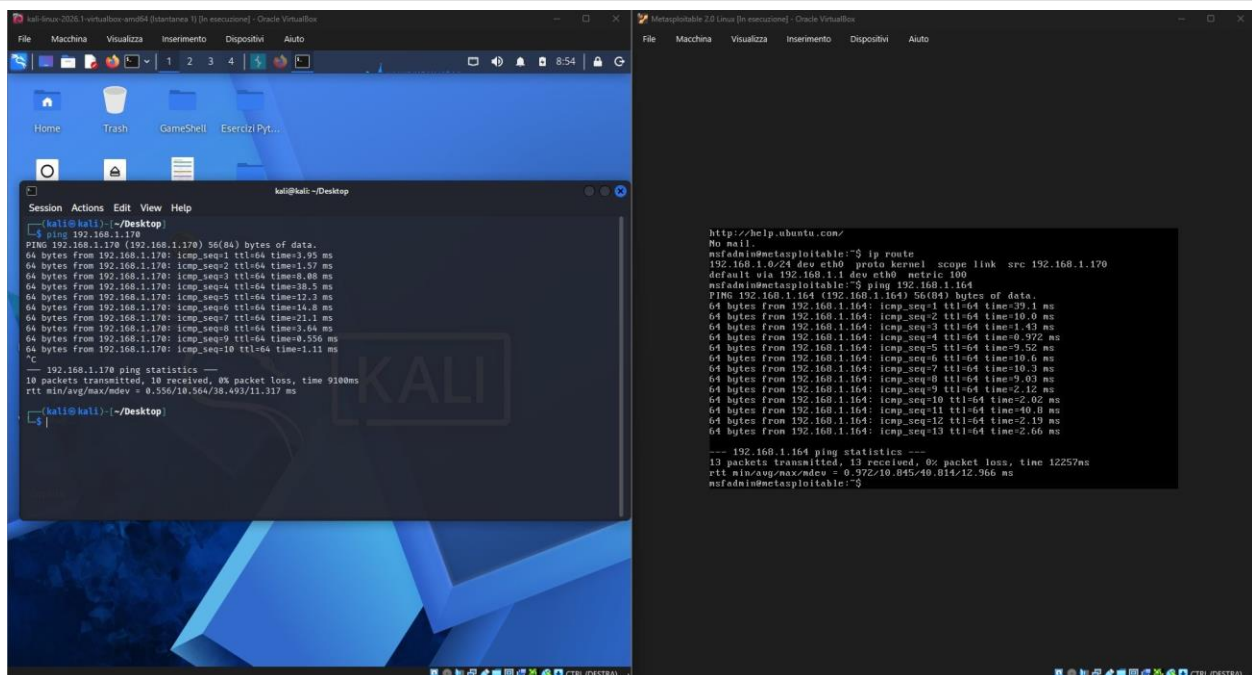


Fig. 1: Ping bidirezionale tra Kali (192.168.1.164) e Metasploitable (192.168.1.170). 0% packet loss in entrambe le direzioni e TTL=64 confermano la raggiungibilita' a livello IP.

STEP 2 | Accesso alla DVWA e configurazione del livello di sicurezza

Dal browser Firefox di Kali ho raggiunto la DVWA all'indirizzo `http://192.168.1.170/dvwa/` e mi sono autenticato con le credenziali di default (`admin / password`). Successivamente, dalla sezione *DVWA Security*, ho impostato il livello di sicurezza a **LOW**: questo equivale ad eseguire la versione del codice piu' permissiva possibile, ideale per studiare la vulnerabilita' nella sua forma piu' pulita prima di salire di livello.

Ho inoltre annotato il cookie di sessione `PHPSESSID`, che mi sarebbe servito successivamente per autenticare le richieste di `sqlmap`. Nelle evidenze raccolte dal pannello Storage del browser, il cookie risulta essere `PHPSESSID=693321fb6c67a6a5eb999a4340f33e54` affiancato dal cookie `security=low`.

STEP 3 | Sfruttamento XSS riflesso (DVWA - xss_r)

Nella sezione *XSS reflected* della DVWA il modulo accetta un parametro `name` in *query string* e lo rispedisce direttamente nel corpo della pagina HTML senza alcun escaping. In condizioni normali un utente vi digiterebbe il proprio nome; io invece vi ho inserito un piccolo frammento di JavaScript:

```
http://192.168.1.170/dvwa/vulnerabilities/xss_r/?name=<script>alert('Attento Broder, Attacco XSS!')</script>
```

Inviando la richiesta, il browser di Kali interpreta il contenuto del parametro come codice eseguibile e mostra immediatamente l'alert. Questo dimostra che la pagina riflette il payload senza applicare alcun filtro o codifica delle entita' HTML: in uno scenario reale, un attaccante potrebbe inserire al posto di `alert()` uno script per rubare il cookie di sessione della vittima e inviarlo a un proprio server.

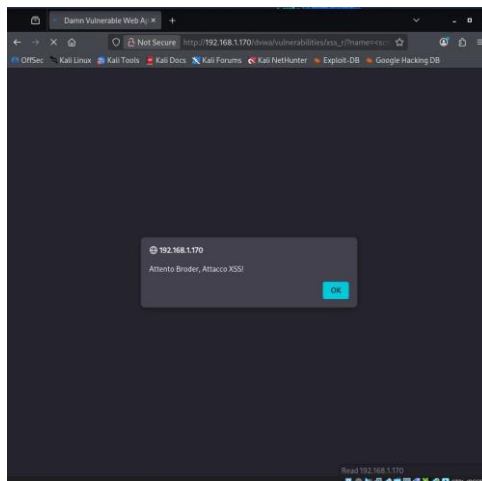


Fig. 2: Esecuzione del payload XSS riflesso: l'URL contiene il tag `<script>` e il browser mostra l'alert "Attento Broder, Attacco XSS!", a riprova che il JavaScript iniettato e' stato eseguito.

STEP 4 | SQL Injection manuale - boolean-based

Nel modulo *SQL Injection* il campo `User ID` viene concatenato direttamente nella query SQL lato server, presumibilmente nella forma `SELECT first_name, last_name FROM users WHERE user_id = '$id'`. Per verificarlo ho inserito come input la classica iniezione tautologica:

```
1' OR '1'='1
```

La query risultante diventa `... WHERE user_id = '1' OR '1'='1'`, condizione sempre vera per definizione. Di conseguenza la DVWA non restituisce piu' soltanto l'utente con `id=1` (admin) ma **l'intera tabella users**: nello screenshot si vedono cinque righe (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith). Questo conferma che il parametro non viene parametrizzato e che la vulnerabilita' e' di tipo SQL Injection in-band.

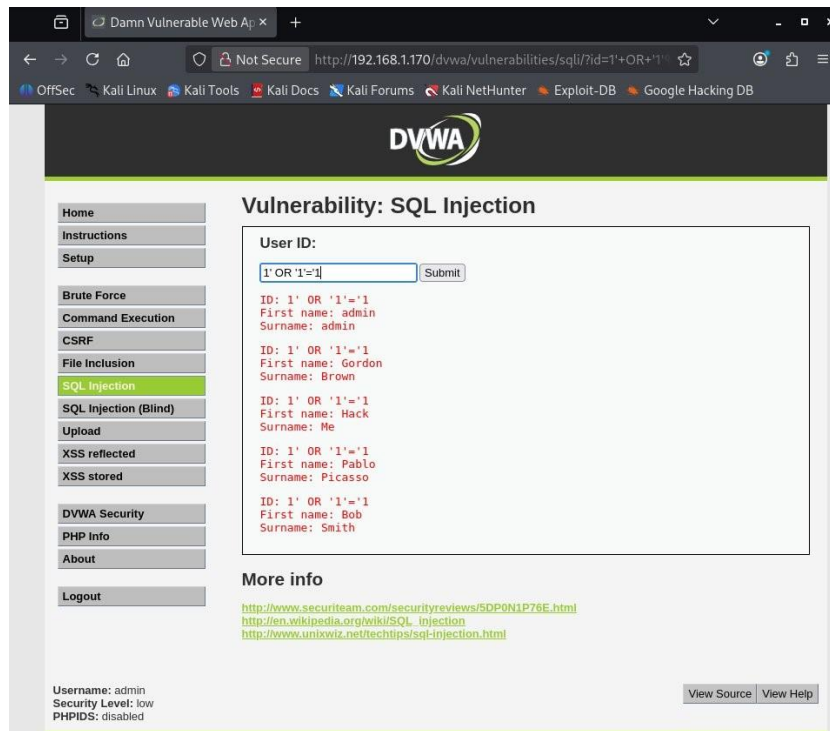


Fig. 3: Iniezione tautologica `1' OR '1'='1'`: la query restituisce l'intera tabella users invece del solo utente richiesto.

STEP 5 | SQL Injection - estrazione credenziali con UNION SELECT

Confermata la presenza dell'iniezione, ho voluto andare oltre alla semplice prova di bypass e provare ad **esfiltrare** informazioni dal database, in particolare la tabella delle credenziali. A questo scopo ho costruito un'iniezione di tipo *UNION-based*:

```
1' UNION SELECT user, password FROM users#
```

Il carattere # in coda commenta tutto cio' che il codice PHP avrebbe aggiunto dopo il mio input, evitando errori di sintassi. La *UNION SELECT* aggancia alla query originaria una seconda estrazione che pesca *user* e *password* dalla tabella *users*. Nel risultato i campi della DVWA vengono valorizzati con il *username* dell'utente del database e con il *password hash* MD5 corrispondente.

Gli hash recuperati sono quelli classici della DVWA: 5f4dcc3b5aa765d61d8327deb882cf99 (admin, smithy), e99a18c428cb38d5f260853678922e03 (gordonb), 8d3533d75ae2c3966d7e0d4fcc69216b (1337), 0d107d09f5bbe40cade3de5c71e9e9b7 (pablo).

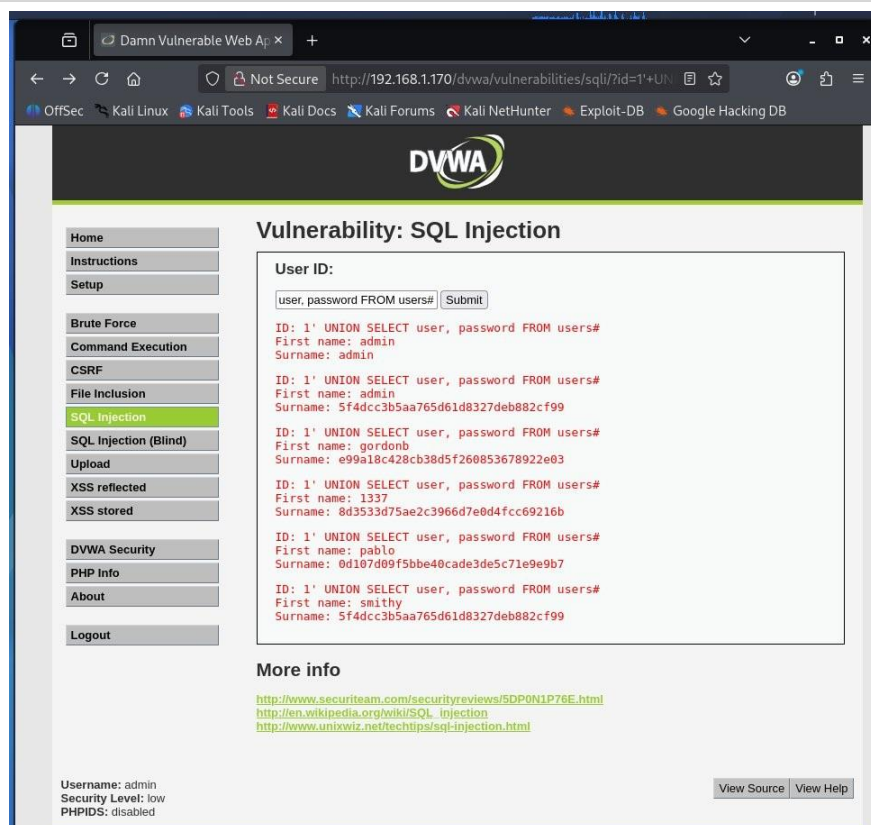


Fig. 4: Risultato dell'iniezione UNION-based: i campi First name e Surname mostrano i valori reali della tabella users (username e password hash MD5).

STEP 6 | Automazione con sqlmap e cracking degli hash

Per concludere ho replicato la stessa estrazione in modo **automatizzato**, usando sqlmap. Lo strumento si occupa di individuare il punto iniettabile, scegliere la tecnica migliore (boolean / UNION / error / time-based) e produrre direttamente il dump della tabella. Trattandosi di un'area dietro login, ho passato a sqlmap il cookie di sessione che avevo annotato in precedenza:

```
sqlmap -u "http://192.168.1.170/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=693321fb6c67a6a5eb999a4340f33e54; security=low" \
-p id --dbms=MySQL -D dvwa -T user --dump-all
```

I flag indicano rispettivamente: `-u` URL target, `--cookie` sessione autenticata, `-p id` parametro da testare, `--dbms=MySQL` DBMS presunto, `-D dvwa -T user` database e tabella da colpire, `--dump-all` per scaricare l'intero contenuto. Al termine sqlmap ha proposto di sottoporre gli hash a un attacco a dizionario (rispondendo `y`) e ha recuperato in chiaro le password `password`, `abc123`, `charley` e `letmein`: tutte parole molto comuni, presenti in qualunque wordlist.

```
(kali@kali)-[~/output/192.168.1.170/dump/dvwa]
$ sqlmap -u "http://192.168.1.170/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=693321fb6c67a6a5eb999a4340f33e54; security=low" -p id --dbms=MySQL -D dvwa -T user --dump-all

{1.10.4#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 11:00:28 /2026-05-12/

[11:00:28] [INFO] testing connection to the target URL
```

Fig. 5: Avvio di sqlmap 1.10.4 contro l'endpoint `sqli/?id=1` della DVWA, con cookie di sessione PHP per superare l'auth.

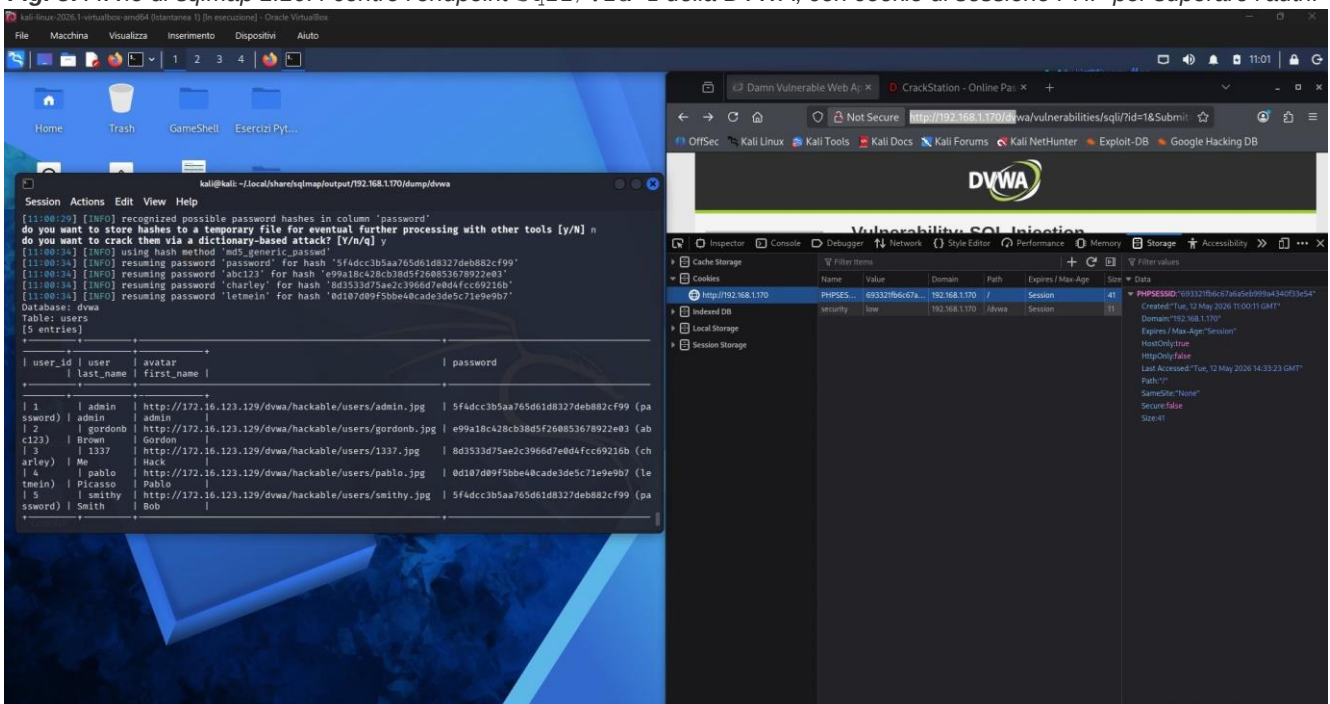


Fig. 6: Output finale di sqlmap: tabella users dumpata con 5 entry e password ricavate via attacco a dizionario (password, abc123, charley, letmein).

5. Appendice: payload e comandi utilizzati

Riporto qui in forma compatta tutti i payload e i comandi significativi impiegati nell'esercitazione, in modo da poterli riprodurre rapidamente in una sessione futura.

Verifica di rete

```
# da Kali ping
192.168.1.170 # da
Metasploitable ip
route ping
192.168.1.164
```

Payload XSS riflesso

```
http://192.168.1.170/dvwa/vulnerabilities/xss_r/?name=<script>alert('Attento Broder, A
ttacco XSS!')</script>
```

Payload SQL Injection (boolean-based)

```
1' OR '1'='1
```

Payload SQL Injection (UNION-based)

```
1' UNION SELECT user, password FROM users#
```

Comando sqlmap completo

```
sqlmap -u "http://192.168.1.170/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
-cookies="PHPSESSID=693321fb6c67a6a5eb999a4340f33e54; security=low" \
-p id --dbms=MySQL -D dvwa -T user --dump-all
```

6. Risultati

Tutti gli obiettivi prefissati dall'esercitazione sono stati raggiunti. Sintetizzo qui sotto i risultati conseguiti.

✓ Comunicazione di rete bidirezionale tra Kali (192.168.1.164) e Metasploitable (192.168.1.170) verificata via ping con 0% packet loss.
✓ DVWA accessibile via Firefox da Kali, autenticazione completata e livello di sicurezza correttamente impostato a LOW.
✓ Vulnerabilita' XSS riflesso confermata: il payload <code><script>alert(...)</script></code> viene eseguito nel browser.
✓ SQL Injection boolean-based dimostrata: con <code>1' OR '1'='1</code> ottengo l'estrazione completa della tabella users.
✓ SQL Injection UNION-based portata a termine con successo: estratti tutti i 5 hash MD5 delle password presenti nella tabella users.
✓ Replica automatica con sqlmap 1.10.4 conclusa correttamente: dump della tabella users e cracking via dizionario di password, abc123, charley, letmein.

7. Difficolta' incontrate

Riporto onestamente i punti in cui mi sono bloccato o ho dovuto tornare sui miei passi, perche' fanno parte del percorso di apprendimento tanto quanto i successi.

- 1. Configurazione di rete.** La prima volta avevo lasciato la scheda di rete della Metasploitable in modalita' NAT, e il ping da Kali falliva sistematicamente. Ho dovuto passare entrambe le VM in bridged adapter e rilasciare/rinegoziare l'IP via `dhclient` per ottenere indirizzi coerenti sulla 192.168.1.0/24.
- 2. Cookie per sqlmap.** Al primo tentativo avevo lanciato sqlmap senza passare il cookie PHPSESSID e lo strumento veniva rispedito alla pagina di login, restituendo l'errore *no parameter(s) found*. Solo dopo aver letto piu' attentamente la documentazione ho capito che dovevo passare anche il cookie `security=low`, perche' la DVWA usa quel cookie per decidere quale versione del codice eseguire.
- 3. UNION SELECT con numero di colonne sbagliato.** Al primo tentativo avevo provato la UNION con tre colonne (`UNION SELECT 1, 2, 3`), ottenendo l'errore *The used SELECT statements have a different number of columns*. Ho capito che la query originale ne restituiva due (*first_name* e *surname*) e ho riallineato di conseguenza il payload finale.
- 4. Encoding dell'URL.** Quando incollavo il payload XSS con spazi o apici nella barra degli indirizzi, Firefox li ricodificava in `%20` e `%27`: non e' stato un problema, ma all'inizio mi sembrava che l'URL *sparisse* e ho perso un po' di tempo prima di rendermi conto che funzionava lo stesso.

8. Conclusioni

L'esercitazione mi ha permesso di toccare con mano due classi di vulnerabilita' che, pur essendo note da oltre vent'anni, restano tra le piu' diffuse nelle applicazioni web reali e occupano stabilmente le prime posizioni della OWASP Top 10. La parte piu' istruttiva, dal mio punto di vista, non e' stata tanto l'esecuzione dei singoli payload, quanto il **riconoscere** nel codice originario della DVWA *la stessa identica forma* che si ritrova in molti progetti reali: concatenazione di stringhe nell'SQL, output dei parametri della query string senza escaping, fiducia implicita nei dati ricevuti dal client.

Sul piano difensivo i due attacchi suggeriscono contromisure diverse ma sovrapponibili: per la SQL Injection l'uso sistematico di **prepared statement** e **query parametrizzate** (PDO, mysqli prepared, ORM), oltre a un principio di privilegio minimo sull'utente del database; per la XSS l'**escaping contestuale** dell'output (HTML, attributi, JavaScript, URL) e una **Content Security Policy** restrittiva che limiti le sorgenti di script ammesse. Entrambe le difese sono rese piu' robuste da una validazione dell'input lato server basata su whitelist.

Mi rendo conto inoltre di quanto sia importante non fermarsi al primo livello di sicurezza: nel prossimo lab ho intenzione di ripetere gli stessi attacchi sui livelli *medium* e *high* della DVWA per osservare quali filtri vengono introdotti, come provano a fermarmi e in che modo si possono (eventualmente) aggirare.